

Projekttag: Chat-Room-Frontend

Elias Tappeiner, David Gstir

Die Idee

Ihr baut heute eine eigene Chat-Anwendung – vollständig im Browser, mit HTML, CSS und JavaScript.

Das Ziel ist eine Webseite, auf der man:

- alle vorhandenen **Chat-Räume** sehen kann,
- einen **neuen Raum erstellen** kann,
- einen Raum **betreten** und die darin gesendeten **Nachrichten lesen** kann,
- und selbst eine **Nachricht schreiben und abschicken** kann.

Der Backend-Server läuft auf dem Schule-Server und ist über das Schulnetzwerk erreichbar. Alle nutzen **denselben Server** – Räume und Nachrichten, die jemand anderes erstellt, sind also sofort für alle sichtbar, sobald die Implementierung funktioniert.

Das Frontend – also alles, was im Browser läuft – baut ihr selbst. HTML-Struktur, CSS-Design und die gesamte JavaScript-Logik sind eure Aufgabe. Wie die Seite aussieht, ist euch überlassen. Wichtig ist, dass sie funktioniert.

Technisches Setup

Legt einen neuen Ordner (z. B. `chatroom`) an mit den Dateien `index.html`, `style.css` und `script.js`.

Der Backend-Server läuft bereits als Docker-Container. Die Basis-URL lautet:

```
http://10.100.120.111:3000
```

Speichere diese URL als Konstante ganz oben in deiner `script.js`, damit du sie nur an einer einzigen Stelle anpassen musst.

Die API

Datenformate

Ein **Raum** sieht immer so aus:

Alle Antworten des Servers sind im **JSON-Format**. Ein **Raum** hat immer folgende Felder:

```
{
  "id": "1",
  "name": "Allgemein",
}
```

```
"createdAt": "2026-05-08T08:15:00.000Z"
}
```

Eine **Nachricht** hat folgende Felder:

```
{
  "id": "7",
  "author": "Anna",
  "text": "Hallo zusammen!",
  "createdAt": "2026-05-08T08:17:42.000Z"
}
```

Das Feld `createdAt` ist ein ISO-8601-Zeitstempel. Um daraus eine lesbare Uhrzeit zu machen, kannst du `new Date(createdAt).toLocaleTimeString()` verwenden.

Endpunkt 1 – GET /rooms

Gibt eine Liste aller vorhandenen Räume zurück.

Antwort (Beispiel):

```
[
  { "id": "1", "name": "Allgemein", "createdAt": "2026-05-08T08:00:00.000Z" },
  { "id": "2", "name": "Hausübungen", "createdAt": "2026-05-08T08:05:00.000Z" }
]
```

Ist noch kein Raum vorhanden, ist die Antwort ein leeres Array `[]`.

Endpunkt 2 – POST /rooms

Erstellt einen neuen Raum.

Request-Body (JSON):

```
{ "name": "Mein neuer Raum" }
```

Antwort bei Erfolg (Status 201):

```
{ "id": "3", "name": "Mein neuer Raum", "createdAt": "2026-05-08T08:20:00.000Z" }
```

Antwort bei Fehler (Status 400):

```
{ "error": "body must contain a non-empty 'name' string" }
```

Endpunkt 3 – GET /rooms/:roomId/messages

Gibt alle Nachrichten eines bestimmten Raums zurück. :roomId ist die id des Raums.

Antwort (Beispiel für Raum mit id "1"):

```
[  
  { "id": "1", "author": "Max", "text": "Wer ist da?", "createdAt": "2026-  
05-08T08:16:00.000Z" },  
  { "id": "2", "author": "Anna", "text": "Hallo zusammen!", "createdAt": "2026-05-  
08T08:17:42.000Z" }  
]
```

Gibt es noch keine Nachrichten, ist die Antwort [].

Gibt es den Raum nicht, ist die Antwort (Status 404):

```
{ "error": "room not found" }
```

Endpunkt 4 – POST /rooms/:roomId/messages

Sendet eine neue Nachricht in einen Raum.

Request-Body (JSON):

```
{ "author": "Max", "text": "Das ist meine erste Nachricht." }
```

Antwort bei Erfolg (Status 201):

```
{ "id": "3", "author": "Max", "text": "Das ist meine erste Nachricht.",  
  "createdAt": "2026-05-08T08:25:00.000Z" }
```

Antwort bei Fehler (Status 400):

```
{ "error": "body must contain non-empty 'author' and 'text' strings" }
```

Was ihr umsetzen sollt

Die folgenden Punkte beschreiben, welche Funktionen eure Anwendung haben soll. Ihr könnt sie der Reihe nach abarbeiten – jeder Punkt baut auf dem vorherigen auf.

1. Räume laden und anzeigen

Wenn die Seite geöffnet wird, sollen sofort alle vorhandenen Räume vom Server abgerufen und als Liste angezeigt werden. Schreibt dafür eine Funktion (z. B. `ladeRaume()`), die ihr beim Start der Seite aufruft.

Falls noch keine Räume existieren, zeigt einen entsprechenden Hinweistext an.

2. Neuen Raum erstellen

Neben der Raumliste soll es ein Eingabefeld und einen Button geben. Trägt der Benutzer einen Namen ein und klickt auf den Button, wird ein neuer Raum auf dem Server angelegt. Danach aktualisiert sich die Raumliste automatisch.

Schlägt das Erstellen fehl (z. B. weil der Name leer war), zeigt die Fehlermeldung an, die der Server zurückschickt.

3. Raum betreten und Nachrichten lesen

Ein Klick auf einen Raum in der Liste soll einen Nachrichtenbereich einblenden, der zunächst versteckt ist. Dort werden alle Nachrichten dieses Raums angezeigt – mit Autor, Text und Uhrzeit.

Schreibt eine Funktion (z. B. `raumBetreten(raum)`), die den gewählten Raum speichert, den Bereich einblendet und die Nachrichten lädt.

4. Nachricht senden

Im Nachrichtenbereich soll ein Formular mit zwei Feldern sein: eines für den Namen des Absenders, eines für den Text. Nach dem Absenden erscheint die neue Nachricht in der Liste.

Nachrichten automatisch aktualisieren

Wenn Benutzer A eine Nachricht schreibt, soll sie bei Benutzer B erscheinen – ohne dass B die Seite neu laden muss. Dafür gibt es zwei Möglichkeiten:

Variante A: Timer (empfohlen für den Einstieg)

`setInterval(funktion, millisekunden)` ruft eine Funktion **wiederholt** in einem festen Zeitabstand auf und gibt eine **Timer-ID** zurück:

```
const timerId = setInterval(ladeNachrichten, 3000); // alle 3 Sekunden
```

Mit `clearInterval(timerId)` stoppt man den Timer wieder:

```
clearInterval(timerId);
```

Wichtiger Fallstrick: Wechselt der Benutzer den Raum, muss der alte Timer gestoppt werden, bevor ein neuer startet. Sonst laufen mehrere Timer gleichzeitig und laden Nachrichten aus dem falschen Raum.

```
let timerId = null;

function starteAutoRefresh() {
  clearInterval(timerId); // alten Timer stoppen
  timerId = setInterval(ladeNachrichten, 3000); // neuen Timer starten
}
```

Fügt dem Nachrichtenbereich eine Checkbox hinzu (z. B. „Automatisch aktualisieren“). Ist sie aktiviert, startet der Timer; wird sie deaktiviert oder der Raum gewechselt, stoppt er.

Variante B: Long Polling (Bonusaufgabe)

Bonusaufgabe: Long Polling

Was ist Long Polling?

Beim normalen Timer-Polling fragt der Browser alle paar Sekunden an: „Gibt es etwas Neues?“ – Der Server antwortet sofort, auch wenn es nichts Neues gibt. Das ist einfach umzusetzen, aber ineffizient: Die meisten Antworten sind leer.

Beim **Long Polling** dreht man das Prinzip um: Der Client schickt eine Anfrage, und der Server antwortet **erst dann**, wenn tatsächlich eine neue Nachricht vorliegt – oder nach spätestens 30 Sekunden mit einem leeren Array. Der Client schickt sofort wieder eine neue Anfrage. Das Ergebnis ist nahezu Echtzeit-Kommunikation:

Client	Server
--- GET ?since=5 -->	
	(wartet ... keine neuen Nachrichten)
	(wartet ... immer noch keine)
	(neue Nachricht trifft ein!)
<-- [{ id:"6" }] ---	
--- GET ?since=6 -->	← sofort neue Anfrage
	...

Der `since`-Parameter

Damit der Server weiß, welche Nachrichten der Client bereits kennt, gibt es den Parameter `?since=<id>`. Der Server liefert nur Nachrichten zurück, deren `id` **größer** als der angegebene Wert ist. Hat der Client zuletzt die Nachricht mit `id: "5"` gesehen, schickt er `?since=5` und bekommt nur wirklich neue Nachrichten.

`GET /rooms/:roomId/messages?since=<lastMessageId>`

Sind noch keine neuen Nachrichten da, wartet der Server die Verbindung offen (max. 30 Sekunden) und antwortet dann mit `[]`.

Antwortbeispiel (neue Nachricht eingetroffen):

```
[
  { "id": "6", "author": "Ben", "text": "Neue Nachricht!", "createdAt": "2026-05-08T09:01:00.000Z" }
]
```

Das Problem: eine laufende Anfrage abbrechen

Wechselt der Benutzer den Raum, während noch eine Long-Poll-Anfrage offen ist, darf die Antwort nicht mehr verarbeitet werden – sie gehört zum alten Raum. Außerdem soll keine zweite Anfrage gestartet werden, während die erste noch läuft.

Dafür gibt es den **AbortController**. Er erlaubt es, eine laufende **fetch**-Anfrage von außen abzubrechen:

```
// 1. Controller erstellen
const controller = new AbortController();

// 2. Signal beim fetch übergeben
const res = await fetch(url, { signal: controller.signal });

// 3. Anfrage abbrechen (z. B. beim Raumwechsel)
controller.abort();
```

Wird **abort()** aufgerufen, wirft **fetch** einen Fehler mit dem Namen **"AbortError"**. Diesen Fall fängt man mit **try/catch** ab und unterscheidet ihn von echten Netzwerkfehlern:

```
try {
  const res = await fetch(url, { signal: controller.signal });
  // Antwort verarbeiten ...
} catch (e) {
  if (e.name === "AbortError") {
    // Absichtlich abgebrochen – nichts tun
  } else {
    // Echter Fehler (z. B. kein Netz)
    console.error("Netzwerkfehler:", e);
  }
}
```

Umsetzung

Ersetzt den Timer aus Variante A durch Long Polling:

- Speichert die höchste bisher gesehene Nachrichten-ID in einer Variable (Startwert 0). Aktualisiert sie nach jedem Laden der Nachrichten.
 - Schreibt eine Funktion `stopLongPoll()`, die eine laufende Anfrage über `controller.abort()` abbricht. Speichert den aktuellen `AbortController` in einer Variable.
 - Schreibt eine Funktion `starteLongPoll()`, die:
 1. zunächst `stopLongPoll()` aufruft,
 2. einen neuen `AbortController` erstellt,
 3. `GET /rooms/:id/messages?since=<letzteId>` abrufen,
 4. neue Nachrichten ans Ende der Liste anfügt und die gespeicherte ID aktualisiert,
 5. nach der Antwort sofort wieder `starteLongPoll()` aufruft (rekursiv),
 6. bei `AbortError` einfach abbricht,
 7. bei einem echten Netzwerkfehler nach 2 Sekunden erneut versucht (`setTimeout(starteLongPoll, 2000)`).
 - Ruft `starteLongPoll()` auf, nachdem `ladeNachrichten()` beim Raumbetreten abgeschlossen ist.
 - Ruft `stopLongPoll()` beim Raumwechsel auf.
-

Präsentation

Eure Lösung soll am Ende des Projekts kurz präsentiert werden.